

# Guida ai Comandi SQL

## CRUD, JOIN, filtri, query innestate e transazioni

UCBM – Ingegneria Industriale

A.A. 2025/2026

- Organizzare i principali comandi SQL
- Distinguere CRUD sullo schema e CRUD sui dati
- Leggere JOIN, filtri e select innestate
- Introdurre le transazioni come unita' logiche di lavoro
- Collegare SQL ai laboratori REPL e web

# Tre famiglie operative

- Schema: tabelle, attributi, vincoli
- Dati: tuple inserite, lette, aggiornate e cancellate
- Transazioni: conferma o annullamento di insiemi di operazioni

# CRUD sullo schema

- `CREATE TABLE`: crea una relazione
- lettura del catalogo: osserva lo schema disponibile
- `ALTER TABLE`: modifica una struttura
- `DROP TABLE`: elimina una relazione

## Punto chiave

Qui non modifichiamo righe: modifichiamo la forma del database.

# CREATE TABLE

```
CREATE TABLE NoteContatto (  
    id_nota INTEGER PRIMARY KEY,  
    id_contatto INTEGER NOT NULL,  
    testo TEXT NOT NULL,  
    data_nota TEXT NOT NULL,  
    FOREIGN KEY (id_contatto) REFERENCES Contatti(id)  
);
```

# Come leggere CREATE TABLE

- nome della relazione
- attributi e domini operativi
- chiave primaria
- attributi obbligatori
- chiavi esterne

## Leggere lo schema

```
SELECT name  
FROM sqlite_master  
WHERE type = 'table'  
ORDER BY name;
```

### Osservazione

Anche il DBMS contiene tabelle che descrivono le tabelle.

# ALTER e DROP

```
ALTER TABLE Contatti  
ADD COLUMN email TEXT;
```

```
DROP TABLE NoteContatto;
```

- ALTER cambia una struttura esistente
- DROP elimina struttura e dati



- INSERT: crea tuple
- SELECT: legge tuple e produce risultati
- UPDATE: modifica tuple esistenti
- DELETE: cancella tuple

# INSERT

```
INSERT INTO Contatti (id, nome, cognome, telefono)
VALUES (6, 'Elena', 'Bruni', '06-0000000');
```

- crea una nuova riga
- deve rispettare chiavi e vincoli

# SELECT

```
SELECT id, nome, cognome, telefono  
FROM Contatti  
ORDER BY cognome, nome;
```

- FROM: tabella di partenza
- SELECT: colonne visibili
- ORDER BY: ordinamento del risultato

# UPDATE

```
UPDATE Contatti  
SET telefono = '06-1111111'  
WHERE id = 6;
```

## Attenzione

Senza WHERE, l'aggiornamento riguarda tutte le righe.

# DELETE

```
DELETE FROM Contatti  
WHERE id = 6;
```

- elimina righe
- puo' essere bloccato da chiavi esterne
- va letto sempre insieme al filtro

- WHERE telefono IS NULL
- WHERE telefono LIKE '06%'
- condizioni composte con AND, OR, NOT

## Regola

WHERE decide quali righe entrano nel risultato.

# Perche' i JOIN sono centrali

- Il modello relazionale distribuisce dati tra tabelle
- I riferimenti collegano le tuple
- Il JOIN ricostruisce una vista leggibile
- La scelta del tipo di JOIN cambia il risultato

# INNER JOIN

```
SELECT i.nome, i.cognome, s.titolo  
FROM SchedeAllenamento s  
JOIN Iscritti i ON s.id_iscritto = i.id_iscritto;
```

- conserva le righe con corrispondenza
- utile quando il legame deve esistere



# LEFT JOIN

```
SELECT i.nome, i.cognome,  
       COUNT(ex.id_esecuzione) AS numero_esecuzioni  
FROM Iscritti i  
LEFT JOIN Esecuzioni ex ON i.id_iscritto = ex.id_iscritto  
GROUP BY i.id_iscritto, i.nome, i.cognome;
```

- conserva tutti gli iscritti
- conta zero esecuzioni quando il match non esiste

# JOIN multipli

```
SELECT i.nome, i.cognome, s.titolo, e.nome_esercizio
FROM SchedeAllenamento s
JOIN Iscritti i ON s.id_iscritto = i.id_iscritto
JOIN SchedaEsercizi se ON s.id_scheda = se.id_scheda
JOIN Esercizi e ON se.id_esercizio = e.id_esercizio
WHERE s.attiva = 1
ORDER BY i.cognome, i.nome, se.ordine_esecuzione;
```

# Lettura dei JOIN multipli

- 1 parto dalle schede
- 2 recupero l'iscritto
- 3 recupero le righe della scheda
- 4 recupero gli esercizi
- 5 filtro le schede attive
- 6 ordino il risultato

```
SELECT e.nome_esercizio,  
       COUNT(ex.id_esecuzione) AS numero_esecuzioni  
FROM Esercizi e  
LEFT JOIN SchedaEsercizi se ON e.id_esercizio = se.id_esercizio  
LEFT JOIN Esecuzioni ex ON se.id_scheda = ex.id_scheda  
      AND se.ordine_esecuzione = ex.ordine_esecuzione  
GROUP BY e.id_esercizio, e.nome_esercizio;
```

# WHERE e HAVING

- WHERE: filtra righe prima del raggruppamento
- HAVING: filtra gruppi dopo il raggruppamento
- nelle query aggregate non sono intercambiabili

# HAVING

```
SELECT e.nome_esercizio,  
       COUNT(ex.id_esecuzione) AS numero_esecuzioni  
FROM Esercizi e  
LEFT JOIN SchedaEsercizi se ON e.id_esercizio = se.id_esercizio  
LEFT JOIN Esecuzioni ex ON se.id_scheda = ex.id_scheda  
      AND se.ordine_esecuzione = ex.ordine_esecuzione  
GROUP BY e.id_esercizio, e.nome_esercizio  
HAVING COUNT(ex.id_esecuzione) >= 2;
```

# Select innestate

- Una query puo' essere usata dentro un'altra query
- La query interna produce un valore o un insieme
- La query esterna usa quel risultato

# Subquery con IN

```
SELECT nome_esercizio, categoria
FROM Esercizi
WHERE categoria IN (
    SELECT categoria
    FROM Esercizi
    WHERE nome_esercizio IN ('Squat', 'Affondi')
);
```



## Subquery come soglia

```
SELECT i.nome, i.cognome, COUNT(ex.id_esecuzione)
FROM Iscritti i
LEFT JOIN Esecuzioni ex ON i.id_iscritto = ex.id_iscritto
GROUP BY i.id_iscritto, i.nome, i.cognome
HAVING COUNT(ex.id_esecuzione) >= (
    SELECT AVG(conteggio)
    FROM (
        SELECT COUNT(*) AS conteggio
        FROM Esecuzioni
        GROUP BY id_iscritto
    )
);
```

## Definizione pratica

Una transazione e' un blocco di operazioni trattato come una sola unita' logica.

- se tutto va bene: confermo
- se qualcosa va male: annullo
- evito stati intermedi incoerenti

- ➊ creo una nuova scheda
- ➋ inserisco gli esercizi della scheda
- ➌ controllo che tutti i riferimenti siano validi
- ➍ confermo solo se la scheda e' completa

# BEGIN, COMMIT, ROLLBACK

```
BEGIN;
```

```
INSERT INTO Contatti (id, nome, cognome, telefono)  
VALUES (6, 'Elena', 'Bruni', '06-0000000');
```

```
COMMIT;
```

```
ROLLBACK;
```

# Come leggere i comandi

- BEGIN: apre il blocco
- COMMIT: rende definitive le modifiche
- ROLLBACK: annulla le modifiche non confermate

```
BEGIN;  
  
INSERT INTO Esecuzioni (...) VALUES (...);  
  
SAVEPOINT seconda_esecuzione;  
INSERT INTO Esecuzioni (...) VALUES (...);  
  
ROLLBACK TO seconda_esecuzione;  
RELEASE seconda_esecuzione;  
  
COMMIT;
```

- I vincoli definiscono stati ammessi
- La transazione governa il passaggio tra stati
- Insieme proteggono la coerenza applicativa

- dimenticare WHERE in UPDATE o DELETE
- usare JOIN quando serve LEFT JOIN
- usare WHERE al posto di HAVING
- non capire il risultato della query interna
- non usare transazioni per operazioni multi-step



- 1 quali tabelle sono coinvolte?
- 2 quali join ricostruiscono i dati?
- 3 quali filtri agiscono sulle righe?
- 4 ci sono gruppi o aggregazioni?
- 5 ci sono select innestate?
- 6 la query modifica uno stato?

- REPL e web trasformano azioni utente in SQL
- le CRUD applicative diventano INSERT, SELECT, UPDATE, DELETE
- i dettagli delle schede usano JOIN multipli
- le operazioni complesse richiedono transazioni

- ➊ SQL gestisce schema, dati e transazioni
- ➋ JOIN e filtri sono il cuore delle query operative
- ➌ le select innestate permettono condizioni derivate
- ➍ le transazioni proteggono operazioni logiche composte